

H I G H - P E R F O R M A N C E M A C H I N E L E A R N I N G

Lecture · LLM Serving Systems

vLLM, PagedAttention, and Continuous Batching

How to serve trillion-parameter language models without melting the GPU.

Kaoutar El Maghraoui · Columbia University & IBM Research

Spring 2026

What we'll cover today

1

Why LLM Serving Is Hard

Inference economics, autoregressive bottleneck, prefill vs decode

2

The KV Cache

Why it dominates memory, growth dynamics, the fragmentation problem

3

Static → Continuous Batching

Orca (OSDI '22), iteration-level scheduling, selective batching

4

PagedAttention & vLLM

Virtual memory for the GPU, block tables, copy-on-write sharing

5

Beyond classic vLLM

Chunked prefill, prefix caching, speculative decoding, disagg P/D

6

The 2026 Industry Landscape

vLLM · SGLang · TensorRT-LLM · TGI · NVIDIA Dynamo

7

Production Practice

Metrics that matter, deployment patterns, cost models

8

Open Problems

Long context, MoE, agentic workloads, hardware co-design

PART I

Module 1

Why LLM serving is hard

Inference is the new training: 10× cheaper per run, $10^9\times$ more often. We start with the workload, not the algorithm.

Inference is now the bill, not training

\$0.50–3.00

per million input tokens

Per-request margins on inference are the difference between a profitable AI product and a money-losing one.

≈ 400K

GPUs running vLLM (2026)

Open-source serving has scaled like Kubernetes did a decade ago — "Linux of inference."

73%

inference cost cut at Stripe

50M daily API calls served on ⅓ of the previous GPU fleet, and the model didn't change.

Optimization is no longer optional, it's the difference between \$10M and \$2.5M of GPU spend.

A model is trained once. It is served forever.

TRAINING

One run

- weeks to months of compute on 10^3 – 10^4 GPUs
- synchronous, gradient-bound, throughput-only
- loss is the only metric you optimize
- 1 model = 1 training job

INFERENCE

Trillions of runs

- every prompt = one inference job
- asynchronous, latency-bound and throughput-bound
- many SLOs at once: TTFT, inter-token latency (ITL), p99, \$/1M tokens
- 1 model serves millions of distinct users with distinct prompt lengths and budgets

The autoregressive bottleneck



each token depends on every previous token $\rightarrow N$ forward passes for N output tokens

Implication 1, depth

Generating 200 tokens means 200 sequential forward passes through 80 layers. Pipelining these passes across requests is the only way to keep tensor cores busy.

Implication 2, state

Each step needs the keys and values from every preceding token. We cannot afford to recompute them, so we cache them, the KV cache. That cache will dominate everything that follows.

Background: autoregressive language modeling, Vaswani et al., "Attention Is All You Need," NeurIPS 2017.

Two phases, two completely different workloads

PREFILL

Compute-bound

Process the entire prompt in one parallel forward pass.

- All N prompt tokens go through every layer at once
- Big GEMMs → tensor cores at 90–95% utilization
- Bottleneck: FLOPs (compute, not memory)
- Quadratic in sequence length
- **Determines TTFT (Time to First Token)**

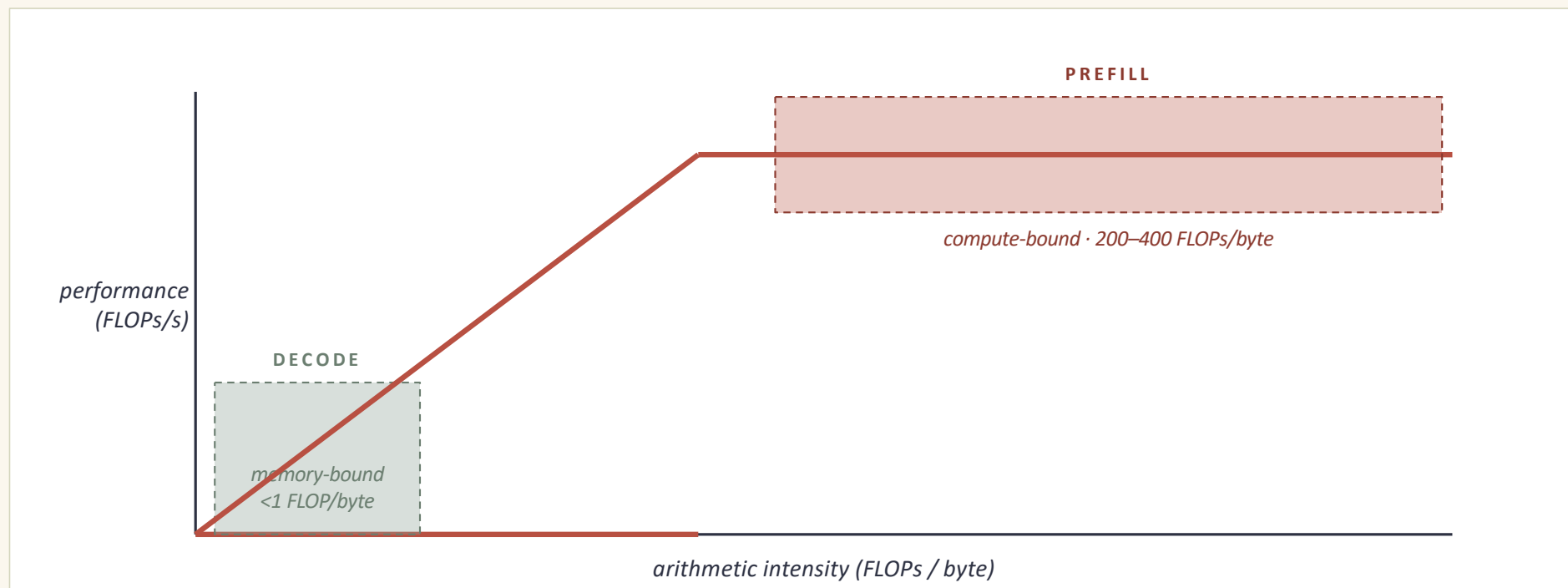
DECODE

Memory-bound

Generate one token at a time, sequentially.

- Only 1 new query token per step → tiny GEMV
- Streams the full KV cache from high-bandwidth memory (HBM) each step
- Bottleneck: memory bandwidth, not compute
- Tensor cores often <10% utilized at batch=1
- **Determines ITL / TPOT (per-token latency)**

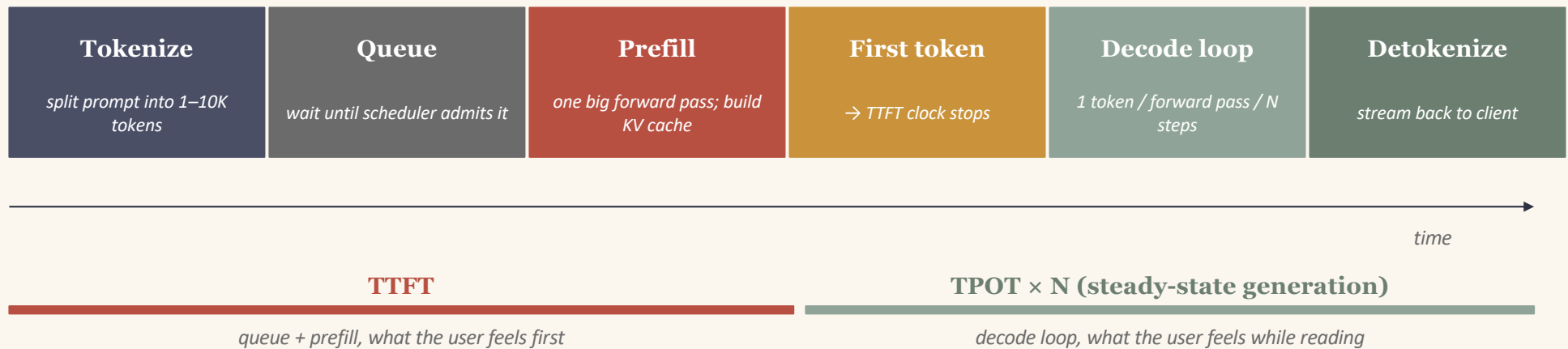
The two phases live on opposite sides of the roofline



On an H100 SXM, prefill achieves 90%+ tensor-core utilization. Decode lives at the memory-bandwidth ceiling, often $<10\%$. One GPU is the wrong shape for either job alone.

Method: Williams et al., "Roofline: An Insightful Visual Performance Model," CACM 2009.

What happens when a user hits Enter



Every optimization in this lecture targets either prefill (TTFT) or decode (TPOT). Continuous batching attacks both. PagedAttention enables both. Disaggregation specializes hardware to each.

THE SERVING SYSTEM HAS TO HANDLE ALL OF THESE

There is no "the LLM workload"

Chat

in: **200–4K**
out: **100–500**
SLO: low TTFT, smooth ITL

stream readability matters most

Code completion

in: **500–8K**
out: **20–200**
SLO: very low TTFT (<200 ms)

user is mid-keystroke

Long-doc RAG

in: **8K–128K**
out: **200–2K**
SLO: TTFT looser, throughput high

prefill dominates

Agentic loops

in: **20K+**
out: **thousands**
SLO: fan-out throughput

many small calls, repeated context

Batch / offline

in: **any**
out: **any**
SLO: \$ per token only

no latency target, pack the GPU

Reasoning models

in: **1–2K**
out: **10K–100K**
SLO: TPOT × N is huge

decode dominates the budget

Workload variance is the real challenge

Prompt length varies 100×

200 to 200,000 tokens in the same minute. Memory and compute budgets shift accordingly.

Output length is unknown

Until the model emits `</s>`, the scheduler has no idea how long this request will run.

Arrivals are bursty

Real traffic is Poisson at best, often log-normal, bursts are 5–10× the mean.

Some requests share context, some don't

System prompts, RAG documents, agent histories, opportunities for KV reuse vary by tenant.

Service-level objectives (SLOs) differ per tenant

An interactive chat needs 50 ms ITL; a nightly batch only needs \$/M tokens. Same model.

Models keep changing

Llama → Qwen → DeepSeek → MoE → reasoning. Yesterday's tuning is wrong today.



MEMORY RULES EVERYTHING AROUND ME

Module 2

The KV cache

The KV cache is the working set of every modern LLM inference engine.

Where the KV cache comes from

For each layer, attention computes:

$$\text{Attn}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V$$

$Q, K, V = X \cdot W_Q, X \cdot W_K, X \cdot W_V$ where X is the token embedding sequence

Naive (re-compute everything)

Each new token re-projects all prior tokens to K and V , then re-attends over them.

Cost to generate token N :

$O(N \cdot d^2)$ per step $\rightarrow O(N^2 \cdot d^2)$ for the full sequence

Quadratic in sequence length. Unusable past a few hundred tokens.

Cached (compute once, reuse)

Stash K and V for every prior token. New token only computes its own K, V, Q .

Cost to generate token N :

$O(N \cdot d)$ per step $\rightarrow O(N^2 \cdot d)$ for the full sequence

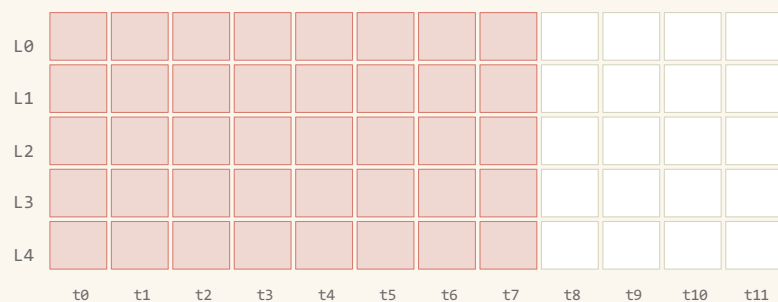
We trade FLOPs for memory. The price is a giant per-request data structure.

Why is decode still slow if attention is cheap? Because you have to READ the whole cache every step. The compute is fine. The bandwidth is the bottleneck

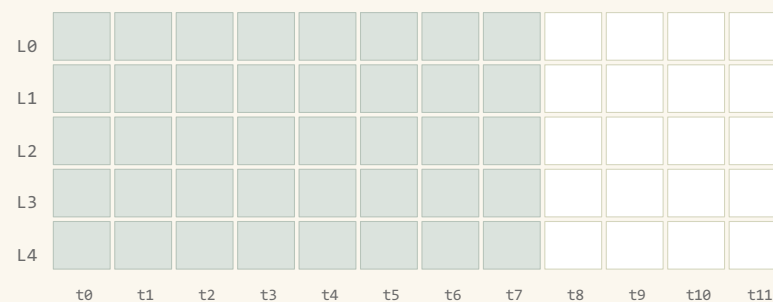
What the KV cache actually is

Two stacks of (layers × tokens) tensors. One for K, one for V. Every layer keeps its own copy.

K cache



V cache



filled, token already seen

empty, capacity reserved but unused

Three things to remember

Per-layer. Every transformer block stores its own K and V tensors. A 70B model has ~80 layers.

Per-head. Multi-head attention multiplies the cost by the number of K/V heads (until grouped-query attention, GQA, and multi-query attention, MQA).

Append-only. Decode writes one new token slice per step; prior slices are immutable for that request.

Where GPU memory actually goes

13B model on a 40 GB A100, typical pre-vLLM allocation

60–80 % of all KV memory is fragmentation waste (Kwon et al., SOSP '23)



Each request reserves a `max_seq_len` slab upfront; almost none fills it.

Model weights	13.2 GB · 33%
Activations	2.0 GB · 5%
KV cache (in use)	12.0 GB · 30%
KV cache (reserved, never filled)	8.0 GB · 20%
Other / overhead	4.8 GB · 12%

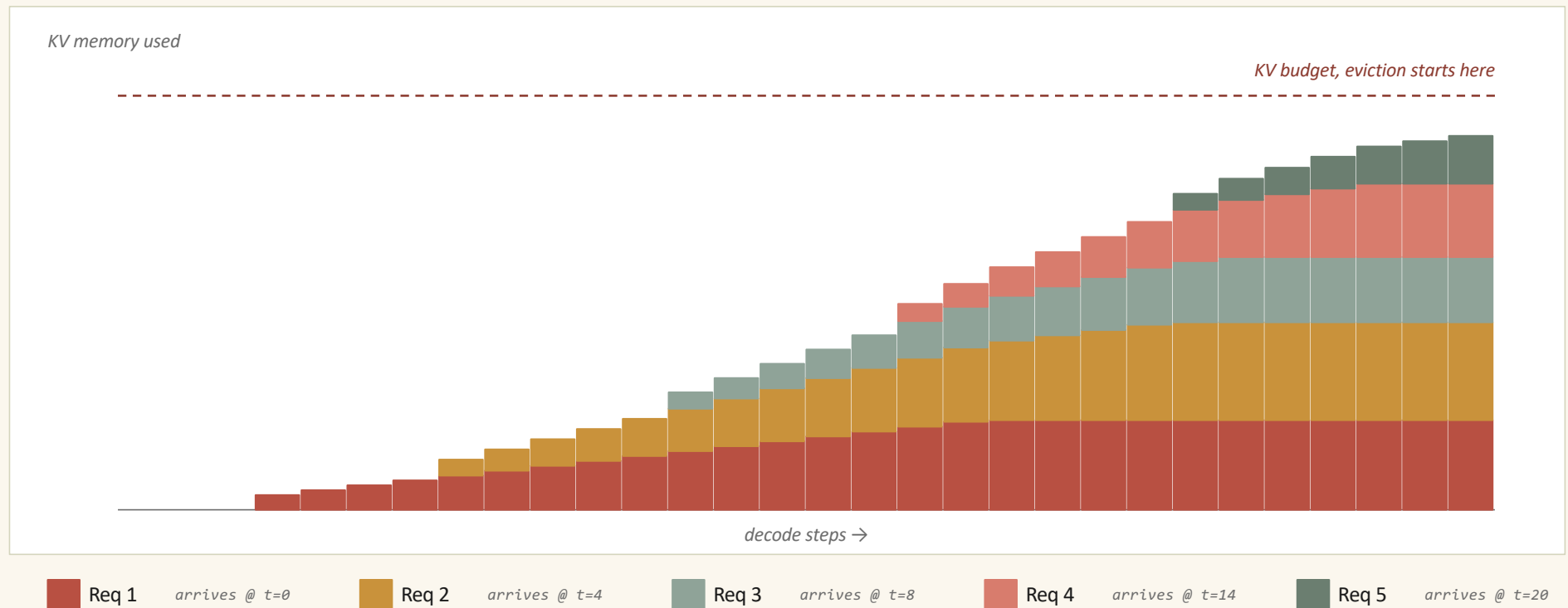
The serving thesis

If you reclaim KV waste, you double effective batch size, and roughly double tokens-per-second per GPU.

Paged KV Motivation

The cache grows token by token

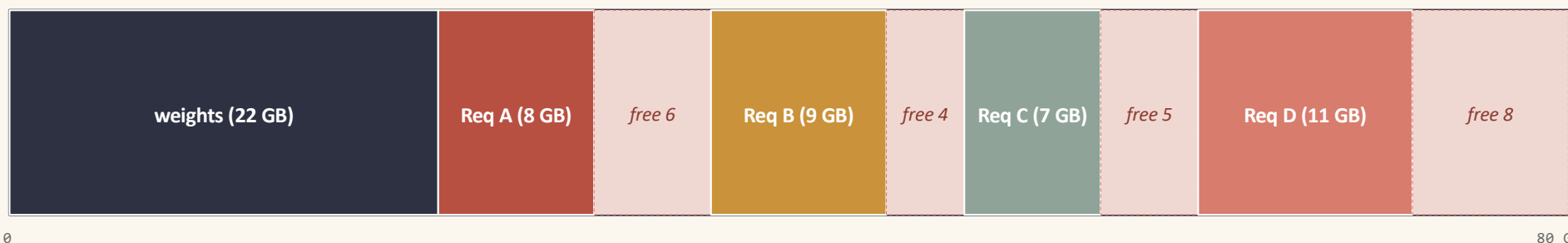
Five concurrent requests, decoding at different rates. Total KV memory grows until the budget is hit.



What fragmentation looks like in HBM

80 GB H100 HBM, mid-day production serving, five live requests, three completed. New 12 GB request arriving.

80 GB HBM



23 GB free in total, but split across four gaps

Incoming: Req E (12 GB)

Largest contiguous gap is 8 GB. Allocator says: out of memory.

Even though 23 GB are technically free.

→ Queue Req E. Throughput drops. Tail latency spikes.

What we want

A KV allocator that:

- hands out small blocks, not slabs
- is OK with non-contiguous memory
- gives back blocks the moment a token leaves
- shares blocks between requests when possible

Architectural Remedies

Before paging, model architects already paid for KV in Q-head budget. Modern models share K/V across query heads, or compress them entirely.

MHA

Multi-Head Attention

Q heads



K/V heads



KV cost

1×

GPT-2, OPT,
Llama-1, GPT-3

GQA

Grouped-Query Attention

Q heads



K/V heads



KV cost

1/4×

Llama-3, Mistral,
Granite, Qwen-2 < 14B

MQA

Multi-Query Attention

Q heads



K/V heads



KV cost

1/8×

Falcon, PaLM,
starter MQA papers

MLA

Multi-head Latent Attention

Q heads



K/V heads



KV cost

1/64×

DeepSeek-V2 / V3
(latent $r=512$)

Architectural and systems-level KV reductions stack. GQA cuts the cache by 4×. PagedAttention recovers another ~2×. MLA closes most of the gap to a query-time-only model.



SCHEDULE THE ITERATION, NOT THE REQUEST

Module 3

Continuous batching

How Orca (OSDI '22) reorganized the unit of scheduling, and gave us 36× higher throughput before anyone touched the kernel.

Static batching wastes the GPU

Four requests with very different output lengths share a static batch. The GPU runs at full width only until the shortest request finishes.



idle slots (GPU keeps running, but for nothing)

Useful work: 26 slots (colored slots). Total slots: 56. **GPU utilization $\approx 26/56 \approx 46\%$.**

Production traces (Orca paper, OSDI '22) report 35–50% as the typical range. This is what continuous batching attacks.

P = Prefill/ D = Decode

Orca's insight: schedule iterations, not requests

Orca's central thesis: the granularity of scheduling should match the granularity of computation.

Yu, Jeong, Lee, Chun · ORCA: A Distributed Serving System for Transformer-Based Generative Models · OSDI '22
<https://www.usenix.org/system/files/osdi22-yu.pdf>

Request-level scheduling (everything before Orca)

Schedule = pick a set of requests, run them through the model end-to-end, then schedule the next set.

Steps

1. Wait until N requests fill a batch
2. Run prompt processing for all of them
3. Run decode loop until all are done
4. Return responses, free memory
5. Repeat

Problem: tail latency from the slowest sequence dominates.

Iteration-level scheduling (Orca, 2022)

Re-run the scheduler before every transformer iteration. New arrivals join, finished requests leave, without restarting the batch.

Steps

1. Pick a set of requests for this iteration
2. Run one transformer forward pass
3. Append generated tokens; check who finished
4. Re-schedule: drop done, add new arrivals
5. Repeat, every iteration

Result: the GPU is always full, requests come and go freely.

Continuous batching: the picture

Four requests, same lengths as before. Now finished slots are immediately reused by new arrivals.



light = prompt (P) · dark = decode (D) · new requests appear the iteration their predecessor finishes

Useful work: 56 slots. Total slots: 56. **GPU utilization ≈ 100%.**

Eight requests served in the same wall-clock time as four. *Same kernels, same model, only the scheduler changed.*

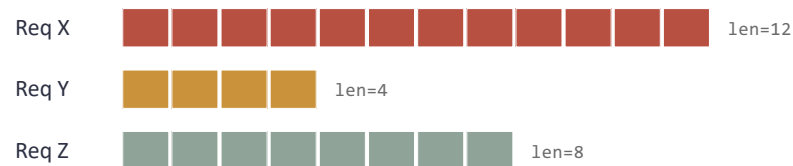
Source: Yu et al., "Orca," OSDI 2022.

Selective batching

Iteration-level scheduling has a problem: requests at different decode steps have different sequence lengths. You can't stack them into one tensor.

The shape mismatch

Three requests at iteration k:



Where this fails:

- Attention: K and V have different lengths per request. Padding wastes compute.
- Batched MatMul: needs uniform sequence dim, won't accept ragged input.
- Layer-norm, FFN: token-wise, actually fine to batch together.

Selective batching

Treat the operations differently:

Token-wise ops (FFN, layer-norm)

Flatten across requests. Batch dim = total tokens in iteration. One giant batched MatMul.

Sequence-aware ops (attention)

Run separately per request, with each request's own K/V tensor. No padding. The 'fused multi-head attention' kernel does many of these in parallel.

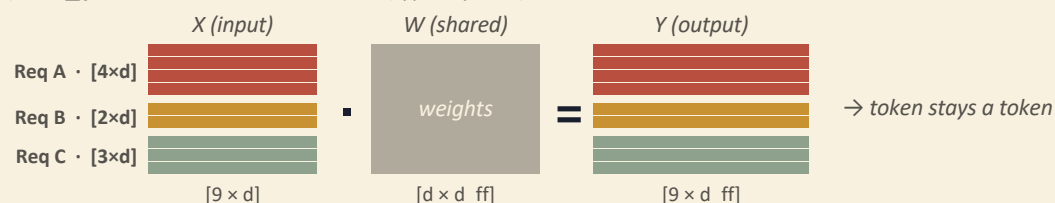
Source: Yu et al., "Orca," OSDI 2022. Section 4 (selective batching).

Why FFN batches but attention doesn't

3 requests at iteration k . Tokens this step: $A = 4$, $B = 2$, $C = 3$. One colored rectangle = one token.

FFN / layer-norm — batchable

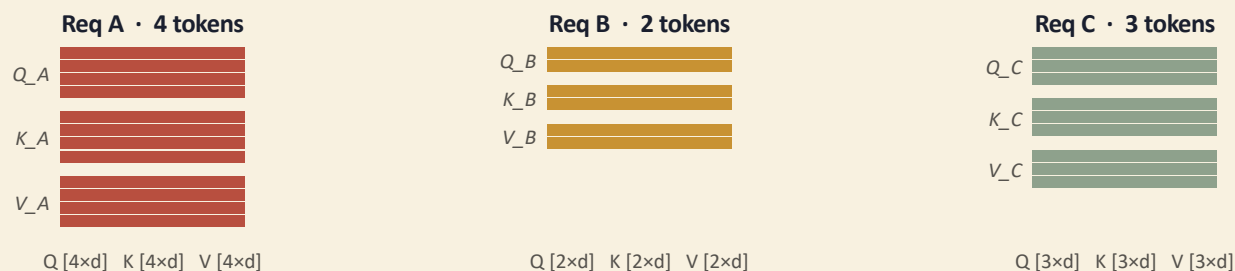
$Y[i,:] = X[i,:] \cdot W$ each output row depends only on its own input row
 where d = hidden size (token vector dimension) · d_{ff} = FFN intermediate dimension (typically $\sim 4d$)



Concatenate 4 + 2 + 3 = 9 tokens into one [9 × d] tensor, do one matmul, split back. Identical to running each separately.

Attention — per-request, ragged

$Y_r = \text{softmax}(Q_r \cdot K_r^T / \sqrt{d}) \cdot V_r$ each request brings its own (Q, K, V) shape



Q_A, Q_B, Q_C have different row counts — they don't fit one matmul.

Pad to max → wasted compute. Concatenate → wrong queries read wrong keys. Solution: 3 attentions, fused into one kernel launch.

How big a deal was this?

36.9×

*throughput vs FasterTransformer
(GPT-3 175B, batch 8)*

1.9×

*latency reduction
at the same throughput*

+2.4×

*effective batch size
at fixed memory budget*

Throughput · GPT-3 13B · NVIDIA A100



Numbers from the paper. The point: a pure scheduling change beat hand-optimized C++ kernels by 36x.



BORROW THE OS PLAYBOOK

Module 4

PagedAttention & vLLM

Kwon et al., SOSP '23. The most-cited LLM systems paper of the decade, and the system that 400,000+ GPUs run today.

The central idea

The vLLM thesis: KV memory should behave like virtual memory, small fixed-size blocks, indirection through a page table, no contiguity required.

OS / virtual memory	KV cache (pre-vLLM)	vLLM (PagedAttention)
Memory split into fixed-size pages (4 KB).	Memory is one giant contiguous slab per request.	Memory split into fixed-size blocks (16 tokens).
Each process has a page table mapping virtual → physical.	Logical position = physical position. No indirection.	Each request has a block table mapping logical → physical block.
Pages allocated on demand, freed when unused.	max_seq_len pre-allocated up front, freed at request end.	Blocks allocated on demand, freed as soon as a request ends.
Internal frag bounded by one page (~4 KB).	Internal frag = max_len – cur_len, often >1 GB.	Internal frag bounded by one block (16 tokens × KV/tok).
Pages can be shared (copy-on-write, COW) between processes.	Each request has its own copy. No sharing.	Blocks can be shared between requests (prefix, beam, parallel).

Blocks: the basic unit of paging

A KV block holds:

- block_size tokens of K and V (default: 16)
- for one layer
- for one request
- stored contiguously in HBM

Block size in bytes (Llama-3 8B GQA):

16 tokens × 8 KV heads × 128 dims × 2 bytes × 2 (K+V)

= 64 KB per block per layer

Why 16?

- Big enough that the indirection overhead is amortized.
- Small enough that internal frag is bounded by ≤ 16 tokens × KV/tok $\approx$ a few MB.
- Aligns with attention kernel tile sizes on H100.

One block · 16 token slots



11 / 16 tokens currently stored

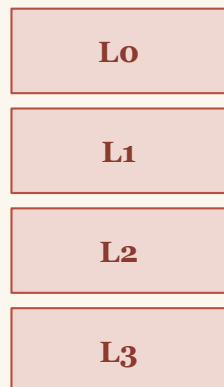
Worst-case waste per request: block_size - 1 = 15 tokens.

Old worst case: max_seq_len - cur_len $\approx$ thousands of tokens.

Logical → physical: the block table

Each request sees its KV cache as a contiguous list of logical blocks. The block table maps each logical block to wherever in HBM it actually lives.

Request A's logical blocks



Block table for Req A

logical	physical	filled
L0	P7	16 / 16
L1	P3	16 / 16
L2	P11	16 / 16
L3	P5	9 / 16

Physical KV pool (in HBM)



Each fetch goes: logical idx → block table[idx] → physical block in HBM.

Source: Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," SOSP 2023.

PagedAttention: the algorithm

```
# attention for a single query token  $q_t$ 
# given block_table for this request, Layer  $\ell$ 

scores = [] # logits for softmax
for logical_block_idx in 0..num_blocks(req):
    phys_idx = block_table[logical_block_idx]
    K_block = kv_pool[ $\ell$ , K, phys_idx] #  $16 \times d_{\text{head}}$ 
    V_block = kv_pool[ $\ell$ , V, phys_idx]

    # only mask out unused slots in the last block
    valid = num_filled_slots(logical_block_idx)
     $s = q_t \cdot K_{\text{block}}[:\text{valid}]^T / \sqrt{d}$ 
    scores.append((s, V_block[:valid]))

p = softmax(concat(scores.s))
out =  $\sum p_k \cdot \text{scores}_k.V$ 
```

Source: Kwon et al., "PagedAttention," SOSP 2023.

Memory accesses

q_t

block_table = [P7, P3, P11, P5]

P7

16 / 16 valid

P3

16 / 16 valid

P11

16 / 16 valid

P5

9 / 16 valid

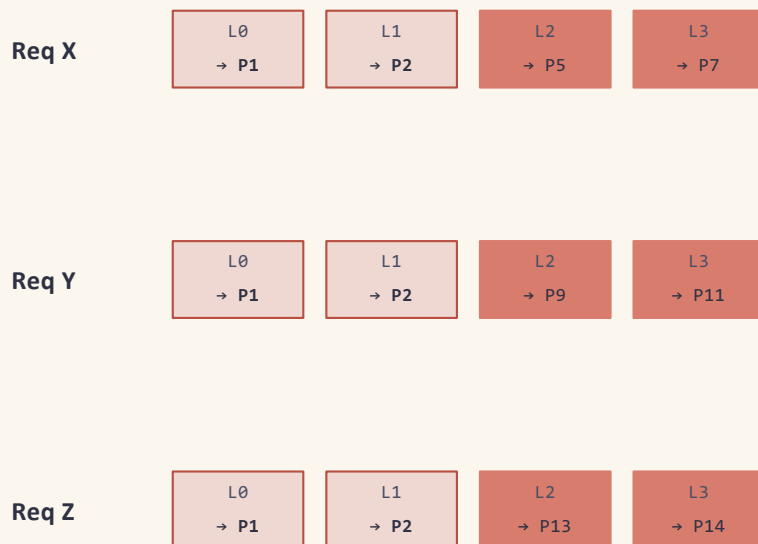
*Four small fetches, scattered.
No contiguity required. Last block partially valid,
masked at compute time.*

Custom CUDA kernel handles the gather.

Block sharing: prefix caching

Many requests share the same prompt prefix (system prompt, RAG context, few-shot examples). With paged KV, you can literally share the blocks.

shared system prompt, one physical copy



Reference counts

Each physical block tracks how many requests are using it. A block is freed only when its refcount drops to zero.

P1 (shared L0)	refcount = 3
P2 (shared L1)	refcount = 3
P5	refcount = 1
P9	refcount = 1
P13	refcount = 1

Result: a 4096-token system prompt costs ~10 MB total instead of $N \times 10$ MB.

In production: 5–10× memory savings for chat-style workloads, reported by LinkedIn and Anthropic.

Source: Kwon et al., "PagedAttention," SOSP 2023. Section 4.3 (block sharing).

The result: fragmentation goes from 60–80% to ~4%

60–80%

wasted KV memory before

<4%

wasted KV memory after

2–4×

throughput improvement on the same hardware

KV memory utilization · LLaMA-13B on A100 80GB · OPT-13B traces

Before · static slabs

30% used

70% wasted

After · PagedAttention

96% used

More usable KV → bigger effective batches → more tokens per second per GPU. Same model. Same hardware. Pure systems.

Module 4 takeaway

1

PagedAttention is virtual memory for KV.

Fixed-size blocks (16 tokens), per-request block tables, allocation on demand. Internal frag bounded by one block.

2

Block sharing turns prefix reuse into bytes saved.

System prompts, RAG context, parallel sampling, beam search, copy-on-write at the block granularity.

3

Fragmentation drops from 60–80% to under 4%.

That alone gives 2–4× throughput on the same hardware. Same model, same kernels, different memory layout.

4

vLLM is the system that built around it.

OpenAI-compatible server, scheduler with admission control + preemption, TP/PP/EP, V1 multi-process engine since 2025.

5

It is the default LLM serving stack in 2026.

400K+ GPUs across Meta, Stripe, Roblox, LinkedIn, IBM, Mistral, Cohere. The vLLM team is now Inferact (\$150M seed, Jan 2026).



SQUEEZING THE REST OF THE LATENCY

Module 5

Beyond classic vLLM

Once paging solved the memory problem, the next round of wins came from rethinking how prefill, decode, and the KV cache itself flow through the system.

Five techniques worth knowing

All five are now default or routinely on in production vLLM / SGLang / TensorRT-LLM deployments.

1

Chunked prefill

Slice long prompts into chunks; co-batch with decode iterations.

vLLM v1 default

2

Automatic prefix caching (APC)

Hash-keyed reuse of KV blocks across requests sharing a prefix.

Free win

3

Speculative decoding

A small draft model proposes K tokens; the target verifies in one pass.

2–4× decode

4

Disaggregated prefill/decode

Run prefill on one pool, decode on another; transfer the KV between them.

DistServe / NIXL

5

KV offload hierarchy

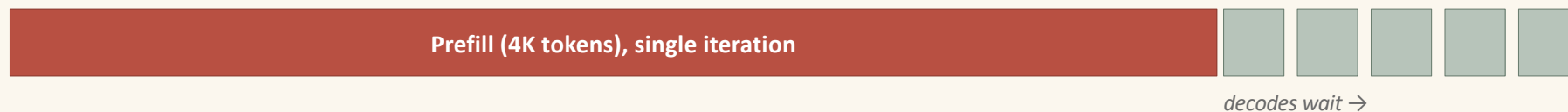
Spill cold KV to CPU RAM, NVMe SSDs, or remote hosts. LMCache, NIXL.

Long-context

Why prefill and decode shouldn't share an iteration

In a naive iteration scheduler, a 4K-token prefill and ten 1-token decodes go through the kernel together. The prefill dominates wall time; the decodes wait. TTFT for new arrivals spikes; TPOT for active streams stalls.

Naive: prefill blocks the iteration



Chunked: prefill is sliced; decodes run with each chunk



Decode latency stays bounded; TTFT improves for new arrivals.

vLLM knobs

- enable-chunked-prefill (default ON in V1)
- max-num-batched-tokens 8192 (good starting point; raise on H100/H200)

Source: Agrawal et al., "SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills," arXiv 2308.16369 (2023).

Automatic prefix caching: the system prompt is free

Most production traffic shares prefixes. System prompts. Few-shot examples. RAG context. Tool schemas. The KV for those tokens does not depend on what comes after, so once it is computed, it can be reused by any later request that starts the same way.



How it works

1. Each KV block keyed by hash(prefix tokens).
2. Block table lookup: hit → reuse, miss → compute.
3. Eviction: LRU when the pool fills.
4. No correctness change, pure caching.

Where it pays off

- Chat with long system prompts (10–60% TTFT cut)
- Few-shot prompting / RAG with stable context
- Agent loops re-using tool schemas every step
- Multi-turn conversations within one session

Source: vLLM Automatic Prefix Caching documentation; Kwon et al., SOSP 2023.

Speculative decoding: why decode wastes the GPU

The decode iteration's arithmetic intensity is < 1 FLOP per byte. A modern GPU has > 200 FLOPs available per byte of HBM bandwidth. The decoder is using less than half a percent of the FLOPs the chip can deliver, every iteration.

The asymmetry

One forward pass on K tokens costs roughly the same wall time as one forward pass on 1 token, both are bandwidth-bound on weight loads. The compute is almost free.

The trick

A small, fast draft model (or a head, or an n -gram) proposes K candidate tokens. The big target model verifies them all in one parallel pass, and because that pass is bandwidth-bound anyway, it costs about the same as one decode step.

The safety

Rejection sampling guarantees the output distribution is exactly the target's. Speculative decoding does not change the model, it changes the schedule. (This matters for evals.)

Source: Leviathan et al., "Fast Inference from Transformers via Speculative Decoding," ICML 2023.



WHERE WE ARE IN 2026

Module 6

LLM serving today

Three open-source engines, one orchestrator, and a market that has consolidated faster than anyone expected.

Four key serving engines

vLLM

UC Berkeley → Inference

Strength · Throughput on heterogeneous workloads. The reference implementation for everything.

Watch out · Single-request latency is not the design target.

Where it runs · OpenAI, Meta, Mistral, Cohere, IBM, Roblox, Stripe, LinkedIn

SGLang

LMSYS / UC Berkeley

Strength · RadixAttention. Excellent on agentic and shared-prefix traffic.

Watch out · Smaller ecosystem; faster-moving API.

Where it runs · xAI, NVIDIA, AMD, LinkedIn, 400K+ GPUs reported

TensorRT-LLM

NVIDIA

Strength · Lowest single-request latency on NVIDIA hardware. AOT-compiled kernels.

Watch out · NVIDIA-only. Per-model build step. Less flexible than vLLM.

Where it runs · Latency-critical paths at NVIDIA-aligned shops

TGI

Hugging Face

Strength · Tight HF Hub integration; the simplest path from a model card to serving.

Watch out · Entered maintenance mode in December 2025. HF now recommends vLLM or SGLang.

Where it runs · Legacy deployments; new ones should migrate.

Pick the engine by the workload

Workload	vLLM	SGLang	TensorRT-LLM
High-QPS chat (mixed lengths)	★ default	good	ok
Agentic / tool-using workloads	good	★ best	ok
Heavy shared-prefix RAG	good (APC)	★ best	ok
Single-request lowest latency	ok	ok	★ best
MoE serving (DeepSeek-V3, Granite 4)	good (V1)	good	★ on Hopper
Long-context (>128K tokens)	★ via offload	good	ok
Multi-LoRA serving	★ default	ok	limited
Edge / non-NVIDIA hardware	★ AMD, AIU	★ AMD	✗ NVIDIA only

★ = recommended starting point. "Good" = production-ready but not the best fit.



OPERATING THE SYSTEM

Module 7

Production practice

Once you've chosen an engine, the work is no longer about kernels. It is about metrics, SLOs, capacity, and what to do at 3 a.m. when P99 explodes.

The four numbers



TTFT

Time to First Token

From client request → first streamed token. Bounded by queue + prefill.

TPOT

Time per Output Token

Mean steady-state inter-token latency during decode. Drives perceived speed.

ITL

Inter-Token Latency

Per-iteration measure. P99 ITL captures stalls TPOT averages away.

E2E

End-to-End Latency

$TTFT + (\text{output_tokens} \times TPOT)$.
What the user actually waits for.

Goodput is the only metric that matters

$$\text{Goodput} = (\text{fraction of requests meeting SLO}) \times \text{throughput}$$

If 10K req/s violates your TTFT SLO on 30% of requests, your true serving capacity is 7K req/s, not 10K.

System A

Throughput · 12 000 req/s
P99 TTFT · 4.2 s (SLO: 1 s)
SLO compliance · 60%

$$\text{Goodput} = 0.60 \times 12\,000 = 7\,200 \text{ req/s}$$

System B

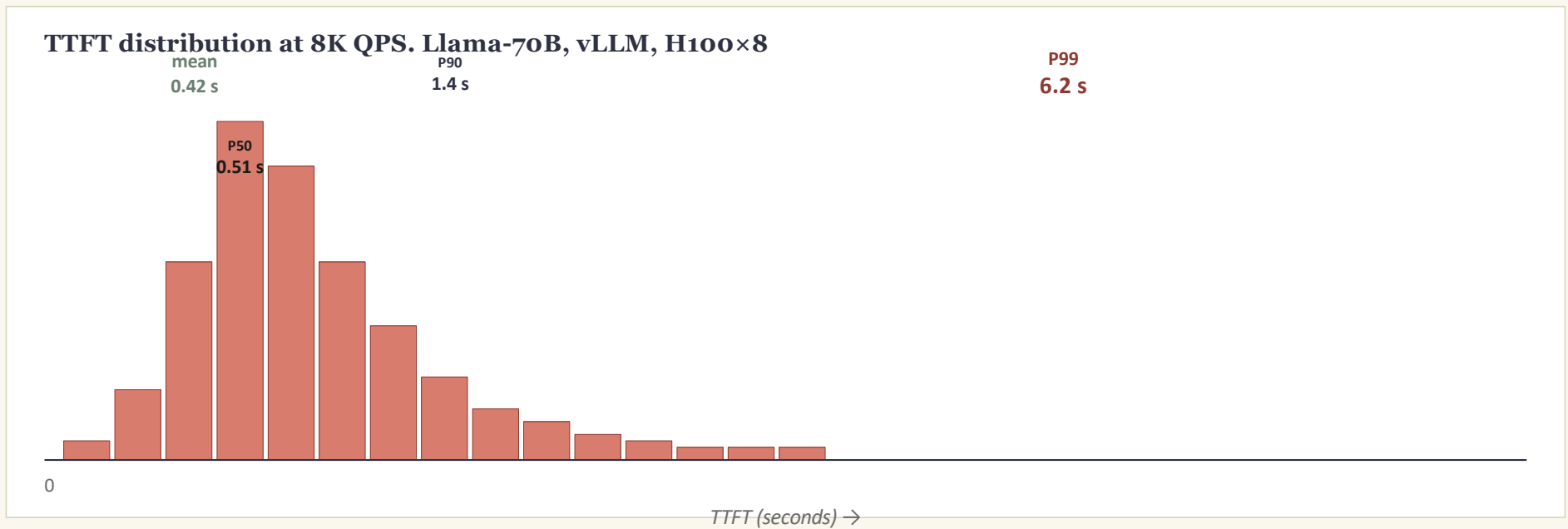
Throughput · 9 000 req/s
P99 TTFT · 0.85 s (SLO: 1 s)
SLO compliance · 99%

$$\text{Goodput} = 0.99 \times 9\,000 = 8\,910 \text{ req/s}$$

System B serves 25% less raw QPS but 24% more goodput. Optimize for what users actually receive.

P99 is the real latency

In serving systems, the worst 1% of requests is what users complain about, what triggers retries, and what cascades into queue blow-ups. The mean is dominated by the easy 99% and tells you nothing about the failure mode.



Eight production pitfalls (and what to do)

Setting `max_model_len` too high

Symptom · KV pool starves; preemption storms.

Fix · Set to your real P99 context, not the model's theoretical max.

Disabling chunked prefill

Symptom · TTFT spikes for new arrivals.

Fix · Leave it on. Tune `max_num_batched_tokens` up, not chunked off.

Treating mean latency as the SLO

Symptom · P99 violations go unseen.

Fix · All dashboards on percentiles. Mean only as a sanity check.

Naive HPA on QPS

Symptom · Scales out into overload.

Fix · Scale on goodput or P95 latency. Custom-metric HPA / KEDA.

Assuming TGI is still maintained

Symptom · Bug fixes never come.

Fix · Migrate to vLLM or SGLang. HF officially recommends both.

Ignoring prefix structure

Symptom · APC hit rate near 0.

Fix · Order prompt: stable system → tools → user. Block-aligned.

Mixing very long + very short reqs

Symptom · Long prefills stall short decodes.

Fix · Separate routing classes; or disaggregate prefill/decode.

No load-shedding plan

Symptom · Queue grows unbounded; everyone times out.

Fix · Set a max queue depth; reject early with 503 + Retry-After.

MODULE 8

Module 8

Where the field is going

Long context, MoE, agentic workloads, hardware co-design, and the assumptions worth questioning.

What's next for serving systems

vLLM-style paged KV with continuous batching is the 2024 consensus. Four pressures are reshaping it.

1. Long context

1M+ token windows break the assumption that KV fits in HBM. Offload, compress, or restructure.

2. Mixture-of-Experts

Sparse activation changes batching arithmetic. Expert parallelism + token routing replaces uniform decode.

3. Agentic workloads

Multi-turn, tool-calling, branching trees. Prefix caching and prompt persistence dominate the system.

4. Hardware co-design

Dataflow accelerators, wafer-scale, custom interconnect. The CUDA monoculture is ending.

Key Insights

1

Inference is the bill.

Trained once, served forever. The serving stack is where dollars and SLOs are decided.

2

Two phases, two workloads.

Prefill is compute-bound, decode is memory-bound. Don't co-batch them naively.

3

The KV cache rules everything.

Capacity, fragmentation, sharing. Every serving advance is a KV-cache advance in disguise.

4

Continuous batching beats static.

Iteration-level scheduling + selective batching. Orca was right; the field followed.

5

PagedAttention enabled the rest.

Block tables turned KV management into virtual memory. vLLM is the reference implementation.

6

Goodput is the metric.

SLO compliance × QPS. Throughput alone is a vanity number.



HPML · SPRING 2026

Thank you.

Questions

Kaoutar El Maghraoui